Company	Squarepoint Capital LLC
Team	Technology → Internal Compute Frameworks (ICF) → Services Core

For context,

Squarepoint , a *global multi-strategy hedge fund*, is split into investment and technology, and technology into teams such as ICF, platform, and data ingestion. Much here is vague due to NDA.

My team built internal-facing cloud platforms. The company makes little use of providers like AWS due to sovereignty and customisation concerns.

My project was a greenfield Python library (with CLI, visualisations, etc.) profiling user code (mostly containers) with caveats: user-level permissions, millisecond resolution, insight generation, and no knowledge of or modifications to user code. It's for individual users locally, and for other ICF platforms to integrate.

1 Onboarding and Beginnings

When interviewing for Squarepoint, I didn't really expect to either pass or like the company; hedge funds have quite poor reputations in my mind, in all but pay and benefits. I ended up with both – my interviewers were personable, quick on their feet, and knowledgeable.

Induction was the part of my experience conformant to a quant firm: a (virtual) stack of documents and one and a half days of training videos. From the get-go, colleagues were very supportive and spent significant time teaching me. I never had lunch alone in the office in six months. Fun fact, due to re-orgs, I steadily migrated closer to the kitchen every two months. *Snacks.*

Squarepoint is the largest company I've worked at, and felt, in the best way possible, a mixture of big-corporate and small-team energy: I can wait weeks for a Jira ticket, but drop meetings into colleagues' calendars in 10 minutes (or just go talk to them).

2 The Work, People, and the Place

When I joined there were around 2,400 employees; now there are 2,800. The dense cloud of specialised tools, processes, and culture was new to me. When I led startups I could design, build, publish, and talk to users in a day. Here, I spend three days researching CPython source code before opening my ticket.

I came to love my project: greenfield, (very?) high potential impact, and free to scope as I saw fit. Over time, more colleagues came to me with questions about my specialisms, and excitement at my tool.

My manager gave me concise, targeted feedback with both opinion and objectivity. I learned that I'm not so good at asking questions, and that questions are as important as my ability to "get it done". It's a subtle lesson I'll keep learning for years.

I settled into a rhythm: office around 09:00, pastries, research and implementation, lunch (provided) and coffee with the team, and leaving at a natural stopping point. It was pressured, since core infrastructure would come to depend on my work, but that was a good motivator. Working from home once a week and flexible hours meant my work mattered and needed to be good, but I still felt like a person. I'd equally settled into a rhythm of being blocked by bureaucracy, often by someone in another timezone; something I've learned to live with, but do not like.

As we deliver (loosely) AWS/GCP equivalents, privacy and security are obvious concerns. My profiler sidesteps most by design: users choose to use it, and requiring user-level permissions and no code modifications vetoed lots of solutions (eBPF, perf, NVIDIA Nsight, and more). It also has a direct environmental impact: my statistics showed half of all memory allocated on one internal platform was never used. I left before my work was fully integrated, and can only hope its been put to good use.

I didn't run into much other issues, partly because my manager (and his manager) shielded me from product managers and corporate nonsense (excuse my French) – a lesson for me and something I hope to imitate one day.

Finance itself is a can of worms: it's great for me and my colleagues, less so for the wider public. A common argument companies make is the liquidity they provide, but honestly, in certain cases its quite easy to pin direct and negative societal impact on financial firms, and not a lot of work produces ever trickles to the public. I don't have a neat answer, and will keep this in mind when I decide where to work in the future.

3 Learnings

Engineering-wise, I'm far improved at Python, Rust, async runtimes, containerisation, CUDA, and more, which easily justifies the placement as integral to DoC's degree.

More important to me are the soft skills: collaboration, building products for real users. These *read* like the headlines for PintOS / WACC and DRP, but the real-life version is very different. My colleagues pushed me to ask more questions, rethink my decisions, and weigh value against effort. My DRP mentors nit-picked about whether CI should run on every commit or just MRs; my manager asks whether supporting MIG devices is worth a week of my time. These are fundamentally different questions, and the latter a natural extension of the former.

Whenever someone asks about Imperial, I sing praises about our focus on real-life engineering: everyone learns databases, but we build :sparkle: for real people :sparkle:. I once managed (not at Squarepoint) some recent graduates who, asked to add CI, promptly asked "what is CI?". Imperial not only taught but (almost) forced me to work in teams with version control, MRs, tickets, and thin slices from user feedback. My manager never had to teach me how to SSH into a dev-box or to open a ticket, so I got more time for the fun stuff.

Personally, I really enjoyed this kind of low-level work, and I'm glad it's not fully replaceable by LLMs, *yet*. It hasn't clearly decided any module choices or career direction for me, but is good experience nonetheless.

One thing I wish DoC had prepared me for is the ubiquity of distributed systems and containerisation. Beyond the usual zeal for microservices, knowing what a container is, or of Kubernetes and SLURM, seems increasingly fundamental rather than a fourth-year elective. In my mind, these would sit very well as a second-year practical, perhaps with a laboratory component.